# 13 *Kernel Machines*

*Kernel machines are maximum margin methods that allow the model to be written as a sum of the influences of a subset of the training instances. These influences are given by application-specific similarity kernels, and we discuss "kernelized" classification, regression, ranking, outlier detection and dimensionality reduction, and how to choose and use kernels.*

## 13.1 Introduction

We now discuss a different approach for linear classification and regression. We should not be surprised to have so many different methods even for the simple case of a linear model. Each learning algorithm has a different inductive bias, makes different assumptions, and defines a different objective function and thus may find a different linear model.

The model that we will discuss in this chapter, called the *support vector machine* (SVM), and later generalized under the name *kernel machine*, has been popular in recent years for a number of reasons:

1. It is a discriminant-based method and uses Vapnik's principle to never solve a more complex problem as a first step before the actual problem (Vapnik 1995). For example, in classification, when the task is to learn the discriminant, it is not necessary to estimate where the class densities $p(\boldsymbol{x}|C_i)$ or the exact posterior probability values $P(C_i|\boldsymbol{x})$; we only need to estimate where the class boundaries lie, that is, $\boldsymbol{x}$ where $P(C_i|\boldsymbol{x}) = P(C_j|\boldsymbol{x})$. Similarly, for outlier detection, we do not need to estimate the full density $p(\boldsymbol{x})$; we only need to find the boundary separating those $\boldsymbol{x}$ that have low $p(\boldsymbol{x})$, that is, $\boldsymbol{x}$ where $p(\boldsymbol{x}) < \theta$, for some threshold $\theta \in (0, 1)$.

2. After training, the parameter of the linear model, the weight vector, can be written down in terms of a subset of the training set, which are the so-called *support vectors*. In classification, these are the cases that are close to the boundary and as such, knowing them allows knowledge extraction: Those are the uncertain or erroneous cases that lie in the vicinity of the boundary between two classes. Their number gives us an estimate of the generalization error, and, as we see below, being able to write the model parameter in terms of a set of instances allows kernelization.

3. As we will see shortly, the output is written as a sum of the influences of support vectors and these are given by *kernel functions* that are application-specific measures of similarity between data instances. Previously, we talked about nonlinear basis functions allowing us to map the input to another space where a linear (smooth) solution is possible; the kernel function uses the same idea.

4. Typically in most learning algorithms, data points are represented as vectors, and either dot product (as in the multilayer perceptrons) or Euclidean distance (as in radial basis function networks) is used. A kernel function allows us to go beyond that. For example, $G_1$ and $G_2$ may be two graphs and $K(G_1, G_2)$ may correspond to the number of shared paths, which we can calculate without needing to represent $G_1$ or $G_2$ explicitly as vectors.

5. Kernel-based algorithms are formulated as convex optimization problems, and there is a single optimum that we can solve for analytically. Therefore we are no longer bothered with heuristics for learning rates, initializations, checking for convergence, and such. Of course, this does not mean that we do not have any hyperparameters for model selection; we do—any method needs them, to match the algorithm to the data at hand.

We start our discussion with the case of classification, and then generalize to regression, ranking, outlier (novelty) detection, and then dimensionality reduction. We see that in all cases basically we have the similar quadratic program template to maximize the separability, or *margin*, of instances subject to a constraint of the smoothness of solution. Solving for it, we get the support vectors. The kernel function defines the space according to its notion of similarity and a kernel function is good if we have better separation in its corresponding space.

## 13.2 Optimal Separating Hyperplane

Let us start again with two classes and use labels $-1/+1$ for the two classes. The sample is $\mathcal{X} = \{\boldsymbol{x}^t, r^t\}$ where $r^t = +1$ if $\boldsymbol{x}^t \in C_1$ and $r^t = -1$ if $\boldsymbol{x}^t \in C_2$. We would like to find $\boldsymbol{w}$ and $w_0$ such that

$$\begin{aligned}\boldsymbol{w}^T\boldsymbol{x}^t + w_0 &\geq +1 \quad \text{for} \quad r^t = +1\\ \boldsymbol{w}^T\boldsymbol{x}^t + w_0 &\leq -1 \quad \text{for} \quad r^t = -1\end{aligned}$$

which can be rewritten as

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \geq +1 \tag{13.1}$$

Note that we do not simply require

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \geq 0$$

Not only do we want the instances to be on the right side of the hyperplane, but we also want them some distance away, for better generalization. The distance from the hyperplane to the instances closest to it on either side is called the *margin*, which we want to maximize for best generalization.

MARGIN

Very early on, in section 2.1, we talked about the concept of the margin when we were talking about fitting a rectangle, and we said that it is better to take a rectangle halfway between $S$ and $G$, to get a breathing space. This is so that in case noise shifts a test instance slightly, it will still be on the right side of the boundary.

Similarly, now that we are using the hypothesis class of lines, the *optimal separating hyperplane* is the one that maximizes the margin.

OPTIMAL SEPARATING HYPERPLANE

We remember from section 10.3 that the distance of $\boldsymbol{x}^t$ to the discriminant is

$$\frac{|\boldsymbol{w}^T\boldsymbol{x}^t + w_0|}{\|\boldsymbol{w}\|}$$

which, when $r^t \in \{-1, +1\}$, can be written as

$$\frac{r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0)}{\|\boldsymbol{w}\|}$$

and we would like this to be at least some value $\rho$:

$$\frac{r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0)}{\|\boldsymbol{w}\|} \geq \rho, \forall t \tag{13.2}$$

We would like to maximize $\rho$ but there are an infinite number of solutions that we can get by scaling $\mathbf{w}$ and for a unique solution, we fix $\rho\|\mathbf{w}\| = 1$ and thus, to maximize the margin, we minimize $\|\mathbf{w}\|$. The task can therefore be defined (see Cortes and Vapnik 1995; Vapnik 1995) as to

$$\min \frac{1}{2}\|\mathbf{w}\|^2 \text{ subject to } r^t(\mathbf{w}^T\mathbf{x}^t + w_0) \geq +1, \forall t \tag{13.3}$$

This is a standard quadratic optimization problem, whose complexity depends on $d$, and it can be solved directly to find $\mathbf{w}$ and $w_0$. Then, on both sides of the hyperplane, there will be instances that are $1/\|\mathbf{w}\|$ away from the hyperplane and the total margin will be $2/\|\mathbf{w}\|$.

We saw in section 10.2 that if the problem is not linearly separable, instead of fitting a nonlinear function, one trick is to map the problem to a new space by using nonlinear basis functions. It is generally the case that this new space has many more dimensions than the original space, and, in such a case, we are interested in a method whose complexity does not depend on the input dimensionality.

In finding the optimal hyperplane, we can convert the optimization problem to a form whose complexity depends on $N$, the number of training instances, and not on $d$. Another advantage of this new formulation is that it will allow us to rewrite the basis functions in terms of kernel functions, as we will see in section 13.5.

To get the new formulation, we first write equation 13.3 as an unconstrained problem using Lagrange multipliers $\alpha^t$:

$$\begin{aligned} L_p &= \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{t=1}^{N} \alpha^t [r^t(\mathbf{w}^T\mathbf{x}^t + w_0) - 1] \\ &= \frac{1}{2}\|\mathbf{w}\|^2 - \sum_t \alpha^t r^t(\mathbf{w}^T\mathbf{x}^t + w_0) + \sum_t \alpha^t \end{aligned} \tag{13.4}$$

This should be minimized with respect to $\mathbf{w}, w_0$ and maximized with respect to $\alpha^t \geq 0$. The saddle point gives the solution.

This is a convex quadratic optimization problem because the main term is convex and the linear constraints are also convex. Therefore, we can equivalently solve the dual problem, making use of the Karush-Kuhn-Tucker conditions. The dual is to *maximize* $L_p$ with respect to $\alpha^t$, subject to the constraints that the gradient of $L_p$ with respect to $\mathbf{w}$ and $w_0$ are 0

and also that $\alpha^t \geq 0$:

$$\frac{\partial L_p}{\partial \mathbf{w}} = 0 \quad \Rightarrow \quad \mathbf{w} = \sum_t \alpha^t r^t \mathbf{x}^t \tag{13.5}$$

$$\frac{\partial L_p}{\partial w_0} = 0 \quad \Rightarrow \quad \sum_t \alpha^t r^t = 0 \tag{13.6}$$

Plugging these into equation 13.4, we get the dual

$$\begin{aligned} L_d &= \frac{1}{2}(\mathbf{w}^T\mathbf{w}) - \mathbf{w}^T \sum_t \alpha^t r^t \mathbf{x}^t - w_0 \sum_t \alpha^t r^t + \sum_t \alpha^t \\ &= -\frac{1}{2}(\mathbf{w}^T\mathbf{w}) + \sum_t \alpha^t \\ &= -\frac{1}{2}\sum_t \sum_s \alpha^t \alpha^s r^t r^s (\mathbf{x}^t)^T \mathbf{x}^s + \sum_t \alpha^t \end{aligned} \tag{13.7}$$

which we maximize with respect to $\alpha^t$ only, subject to the constraints

$$\sum_t \alpha^t r^t = 0, \text{ and } \alpha^t \geq 0, \forall t$$

This can be solved using quadratic optimization methods. The size of the dual depends on $N$, sample size, and not on $d$, the input dimensionality. The upper bound for time complexity is $\mathcal{O}(N^3)$, and the upper bound for space complexity is $\mathcal{O}(N^2)$.

Once we solve for $\alpha^t$, we see that though there are $N$ of them, most vanish with $\alpha^t = 0$ and only a small percentage have $\alpha^t > 0$. The set of $\mathbf{x}^t$ whose $\alpha^t > 0$ are the *support vectors*, and as we see in equation 13.5, $\mathbf{w}$ is written as the weighted sum of these training instances that are selected as the support vectors. These are the $\mathbf{x}^t$ that satisfy

$$r^t(\mathbf{w}^T\mathbf{x}^t + w_0) = 1$$

and lie on the margin. We can use this fact to calculate $w_0$ from any support vector as

$$w_0 = r^t - \mathbf{w}^T\mathbf{x}^t \tag{13.8}$$

For numerical stability, it is advised that this be done for all support vectors and an average be taken. The discriminant thus found is called the *support vector machine* (SVM) (see figure 13.1).

SUPPORT VECTOR MACHINE

The majority of the $\alpha^t$ are 0, for which $r^t(\mathbf{w}^T\mathbf{x}^t + w_0) > 1$. These are the $\mathbf{x}^t$ that lie more than sufficiently away from the discriminant,

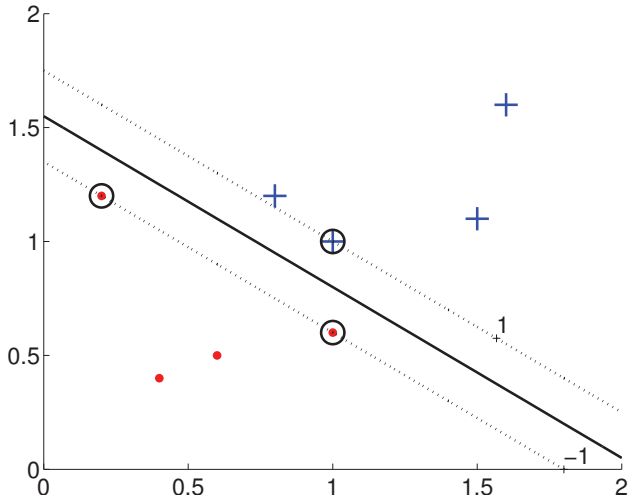


**Figure 13.1** For a two-class problem where the instances of the classes are shown by plus signs and dots, the thick line is the boundary and the dashed lines define the margins on either side. Circled instances are the support vectors.

and they have no effect on the hyperplane. The instances that are not support vectors carry no information; even if any subset of them are removed, we would still get the same solution. From this perspective, the SVM algorithm can be likened to the condensed nearest neighbor algorithm (section 8.5), which stores only the instances neighboring (and hence constraining) the class discriminant.

Being a discriminant-based method, the SVM cares only about the instances close to the boundary and discards those that lie in the interior. Using this idea, it is possible to use a simpler classifier before the SVM to filter out a large portion of such instances, thereby decreasing the complexity of the optimization step of the SVM (exercise 1).

During testing, we do not enforce a margin. We calculate $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0$, and choose according to the sign of $g(\mathbf{x})$:

Choose $C_1$ if $g(\mathbf{x}) > 0$ and $C_2$ otherwise

## 13.3 The Nonseparable Case: Soft Margin Hyperplane

If the data is not linearly separable, the algorithm we discussed earlier will not work. In such a case, if the two classes are not linearly separable such that there is no hyperplane to separate them, we look for the one that incurs the least error. We define *slack variables*, $\xi^t \geq 0$, which store the deviation from the margin. There are two types of deviation: An instance may lie on the wrong side of the hyperplane and be misclassified. Or, it may be on the right side but may lie in the margin, namely, not sufficiently away from the hyperplane. Relaxing equation 13.1, we require

SLACK VARIABLES

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \geq 1 - \xi^t \tag{13.9}$$

If $\xi^t = 0$, there is no problem with $\boldsymbol{x}^t$. If $0 < \xi^t < 1$, $\boldsymbol{x}^t$ is correctly classified but in the margin. If $\xi^t \geq 1$, $\boldsymbol{x}^t$ is misclassified (see figure 13.2). The number of misclassifications is $\#\{\xi^t > 1\}$, and the number of nonseparable points is $\#\{\xi_t > 0\}$. We define *soft error* as

SOFT ERROR

$$\sum_t \xi^t$$

and add this as a penalty term:

$$L_p = \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_t \xi^t \tag{13.10}$$

subject to the constraint of equation 13.9. $C$ is the penalty factor as in any regularization scheme trading off complexity, as measured by the $L_2$ norm of the weight vector (similar to weight decay in multilayer perceptrons; see sectiona 11.9 and 11.10), and data misfit, as measured by the number of nonseparable points. Note that we are penalizing not only the misclassified points but also the ones in the margin for better generalization, though these latter would be correctly classified during testing.

Adding the constraints, the Lagrangian of equation 13.4 then becomes

$$L_p = \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_t \xi^t - \sum_t \alpha^t[r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) - 1 + \xi^t] - \sum_t \mu^t\xi^t \tag{13.11}$$

where $\mu_t$ are the new Lagrange parameters to guarantee the positivity of $\xi^t$. When we take the derivatives with respect to the parameters and set them to 0, we get

$$\frac{\partial L_p}{\partial \boldsymbol{w}} = \boldsymbol{w} - \sum_t \alpha^t r^t \boldsymbol{x}^t = 0 \Rightarrow \boldsymbol{w} = \sum_t \alpha^t r^t \boldsymbol{x}^t \tag{13.12}$$

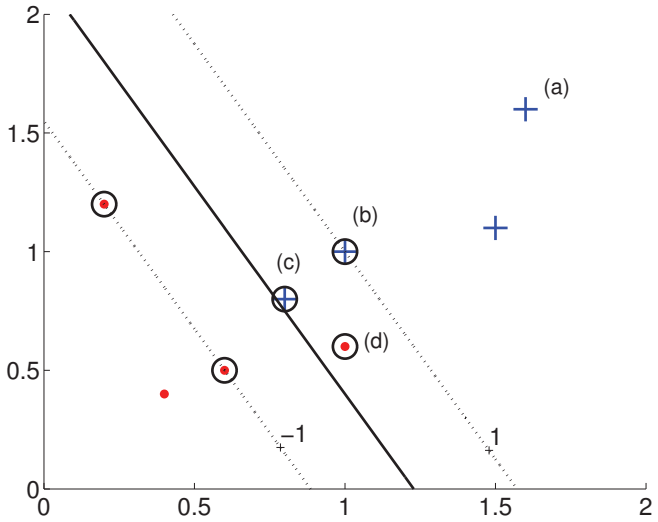


**Figure 13.2** In classifying an instance, there are four possible cases: In (a), the instance is on the correct side and far away from the margin; $r^t g(\mathbf{x}^t) > 1$, $\xi^t = 0$. In (b), $\xi^t = 0$; it is on the right side and on the margin. In (c), $\xi^t = 1 - g(\mathbf{x}^t), 0 < \xi < 1$; it is on the right side but is in the margin and not sufficiently away. In (d), $\xi^t = 1 + g(\mathbf{x}^t) > 1$; it is on the wrong side—this is a misclassification. All cases except (a) are support vectors. In terms of the dual variable, in (a), $\alpha^t = 0$; in (b), $\alpha^t < C$; in (c) and (d), $\alpha^t = C$.

$$\frac{\partial L_p}{\partial w_0} = \sum_t \alpha^t r^t = 0 \tag{13.13}$$

$$\frac{\partial L_p}{\partial \xi^t} = C - \alpha^t - \mu^t = 0 \tag{13.14}$$

Since $\mu^t \geq 0$, this last implies that $0 \leq \alpha^t \leq C$. Plugging these into equation 13.11, we get the dual that we maximize with respect to $\alpha^t$:

$$L_d = \sum_t \alpha^t - \frac{1}{2} \sum_t \sum_s \alpha^t \alpha^s r^t r^s (\mathbf{x}^t)^T \mathbf{x}^s \tag{13.15}$$

subject to

$$\sum_t \alpha^t r^t = 0 \text{ and } 0 \leq \alpha^t \leq C, \forall t$$

Solving this, we see that as in the separable case, instances that lie on the correct side of the boundary with sufficient margin vanish with their $\alpha^t = 0$ (see figure 13.2). The support vectors have their $\alpha^t > 0$ and they define $\mathbf{w}$, as given in equation 13.12. Of these, those whose $\alpha^t < C$ are the ones that are on the margin, and we can use them to calculate $w_0$; they have $\xi^t = 0$ and satisfy $r^t(\mathbf{w}^T\mathbf{x}^t + w_0) = 1$. Again, it is better to take an average over these $w_0$ estimates. Those instances that are in the margin or misclassified have their $\alpha^t = C$.

The nonseparable instances that we store as support vectors are the instances that we would have trouble correctly classifying if they were not in the training set; they would either be misclassified or classified correctly but not with enough confidence. We can say that the number of support vectors is an upper-bound estimate for the expected number of errors. And, actually, Vapnik (1995) has shown that the expected test error rate is

$$E_N[P(error)] \leq \frac{E_N[\#\text{ of support vectors}]}{N}$$

where $E_N[\cdot]$ denotes expectation over training sets of size $N$. The nice implication of this is that it shows that the error rate depends on the number of support vectors and not on the input dimensionality.

HINGE LOSS

Equation 13.9 implies that we define error if the instance is on the wrong side or if the margin is less than 1. This is called the *hinge loss*. If $y^t = \mathbf{w}^T\mathbf{x}^t + w_0$ is the output and $r^t$ is the desired output, hinge loss is defined as

$$L_{hinge}(y^t, r^t) = \begin{cases} 0 & \text{if } y^t r^t \geq 1 \\ 1 - y^t r^t & \text{otherwise} \end{cases} \tag{13.16}$$

In figure 13.3, we compare hinge loss with 0/1 loss, squared error, and cross-entropy. We see that unlike 0/1 loss, hinge loss also penalizes instances in the margin even though they may be on the correct side, and the loss increases linearly as the instance moves away on the wrong side. This is different from the squared loss that therefore is not as robust as the hinge loss. We see that cross-entropy minimized in logistic discrimination (section 10.7) or by the linear perceptron (section 11.3) is a good continuous approximation to the hinge loss.

$C$ of equation 13.10 is the regularization parameter fine-tuned using cross-validation. It defines the trade-off between margin maximization and error minimization: If it is too large, we have a high penalty for nonseparable points, and we may store many support vectors and overfit.

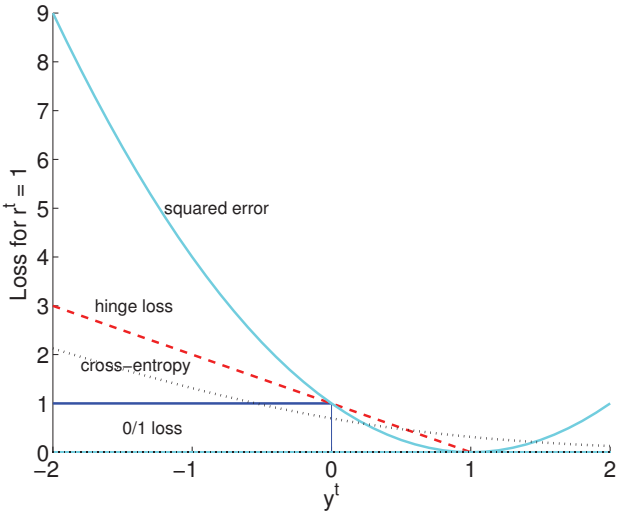


**Figure 13.3** Comparison of different loss functions for $r^t = 1$: 0/1 loss is 0 if $y^t = 1$, 1 otherwise. Hinge loss is 0 if $y^t > 1$, $1 - y^t$ otherwise. Squared error is $(1 - y^t)^2$. Cross-entropy is $\log(1/(1 + \exp(-y^t)))$.

If it is too small, we may find too simple solutions that underfit. Typically, one chooses from $[10^{-6}, 10^{-5}, \ldots, 10^{+5}, 10^{+6}]$ in the log scale by looking at the accuracy on a validation set.

## 13.4 $\nu$-SVM

There is another, equivalent formulation of the soft margin hyperplane that uses a parameter $\nu \in [0, 1]$ instead of $C$ (Schölkopf et al. 2000). The objective function is

$$\min \frac{1}{2}\|\boldsymbol{w}\|^2 - \nu\rho + \frac{1}{N}\sum_t \xi^t \tag{13.17}$$

subject to

$$r^t(\boldsymbol{w}^T\boldsymbol{x}^t + w_0) \geq \rho - \xi^t,\ \xi^t \geq 0,\ \rho \geq 0 \tag{13.18}$$

$\rho$ is a new parameter that is a variable of the optimization problem and scales the margin: The margin is now $2\rho/\|\boldsymbol{w}\|$. $\nu$ has been shown to be

a lower bound on the fraction of support vectors and an upper bound on the fraction of instances having margin errors ($\sum_t \#\{\xi^t > 0\}$). The dual is

$$L_d = -\frac{1}{2}\sum_t \sum_s \alpha^t \alpha^s r^t r^s (\boldsymbol{x}^t)^T \boldsymbol{x}^s \tag{13.19}$$

subject to

$$\sum_t \alpha^t r^t = 0,\ 0 \le \alpha^t \le \frac{1}{N},\ \sum_t \alpha^t \ge \nu$$

When we compare equation 13.19 with equation 13.15, we see that the term $\sum_t \alpha^t$ no longer appears in the objective function but is now a constraint. By playing with $\nu$, we can control the fraction of support vectors, and this is advocated to be more intuitive than playing with $C$.

## 13.5 Kernel Trick

Section 10.2 demonstrated that if the problem is nonlinear, instead of trying to fit a nonlinear model, we can map the problem to a new space by doing a nonlinear transformation using suitably chosen basis functions and then use a linear model in this new space. The linear model in the new space corresponds to a nonlinear model in the original space. This approach can be used in both classification and regression problems, and in the special case of classification, it can be used with any scheme. In the particular case of support vector machines, it leads to certain simplifications that we now discuss.

Let us say we have the new dimensions calculated through the basis functions

$$\boldsymbol{z} = \boldsymbol{\phi}(\boldsymbol{x}) \text{ where } z_j = \phi_j(\boldsymbol{x}), j = 1, \ldots, k$$

mapping from the $d$-dimensional $\boldsymbol{x}$ space to the $k$-dimensional $\boldsymbol{z}$ space where we write the discriminant as

$$\begin{aligned} g(\boldsymbol{z}) &= \boldsymbol{w}^T \boldsymbol{z} \\ g(\boldsymbol{x}) &= \boldsymbol{w}^T \boldsymbol{\phi}(\boldsymbol{x}) \\ &= \sum_{j=1}^{k} w_j \phi_j(\boldsymbol{x}) \end{aligned} \tag{13.20}$$

where we do not use a separate $w_0$; we assume that $z_1 = \phi_1(\boldsymbol{x}) \equiv 1$. Generally, $k$ is much larger than $d$ and $k$ may also be larger than $N$, and there